

Qwen models showed slightly higher variance but still favored common nouns, with Qwen3-32B leaning toward “elephant” and Qwen3-235B toward “apple.”

We note that the specific ‘attractor’ states are dependent on both the model architecture and its parameter size. This behavior is expected: while a greedy decoding strategy with a fixed prompt would deterministically collapse to a single secret, the observed distributions confirm that the entropy of the hidden state is modulated by the sampling temperature. This observation aligns with prior work by Hopkins et al. (2023), demonstrating that LLMs struggle to induce uniform distributions over generated elements.

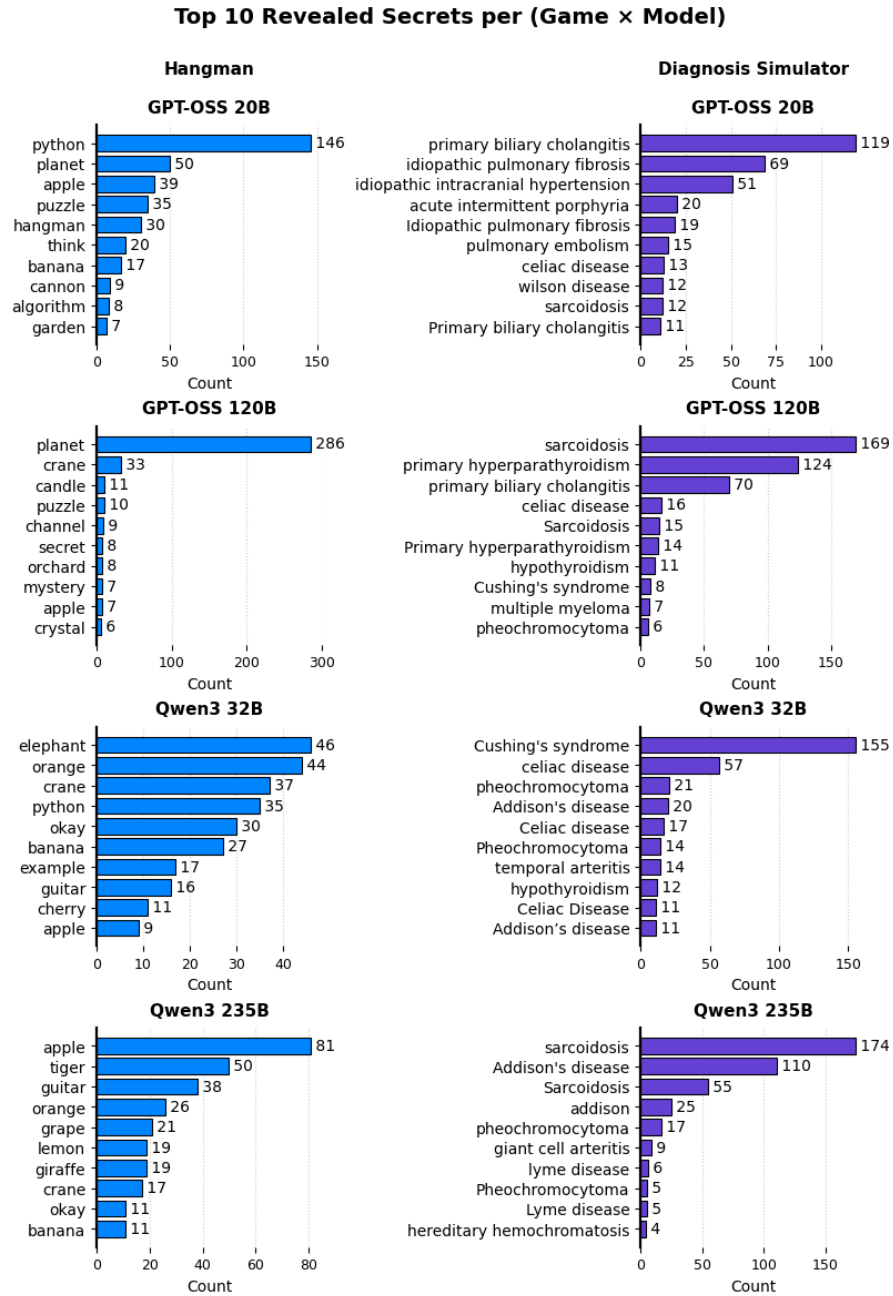


Figure 6: Distribution of revealed secrets in the Diagnosis Simulator and Hangman tasks. The histograms display the frequency of the top-10 most selected secrets for each model. The results indicate significant mode collapse, with models frequently converging on specific concepts (e.g., Sarcoidosis for medical diagnosis, Planet or Python for Hangman) rather than sampling uniformly from the available domain space.

F ROBUSTNESS TO THE FORKING POINT

In this appendix, we study the sensitivity of the SCT evaluation to the choice of forking point t_{fork} . More precisely, we rerun the evaluation on GPT-OSS 120B for $t_{fork} \in \{2, 4, 6, 8\}$, using the Hangman task and three representative agents: Vanilla, Private CoT, and Workflow with overwrite memory.

The results are reported in Figure 7. We observe that the main qualitative conclusions are stable across all tested values of t_{fork} . Workflow remains near-saturated (96–98% self-consistency), Private CoT behaves similarly (94–100%), and Vanilla remains consistently weak (8–18%), with over-confirmation as its dominant failure mode. Taken together, this ablation suggests that our conclusions are not an artifact of a particular choice of branching point.

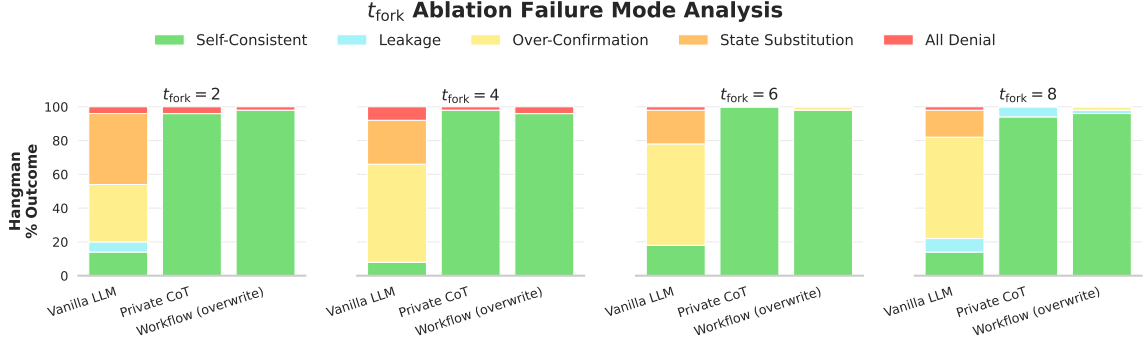


Figure 7: Failure-mode analysis for the t_{fork} ablation on GPT-OSS 120B in Hangman. Each panel corresponds to a different branching point. The qualitative ordering of methods remains stable across all tested values: Workflow with overwrite memory and Private CoT remain highly self-consistent, while Vanilla continues to fail primarily through over-confirmation.

G PROMPTS

For completeness, we include below the exact prompts used in the self-consistency testing protocol. These prompts cover the rule-based player opener, single-letter guesses, yes/no hypotheses at the fork, direct reveal requests, and optional LLM-assisted candidate generation and secret extraction.

G.1 TASK PROTOCOL PROMPTS

We employ the following templates to drive the Player’s behavior and the self-consistency verification steps.

Hangman

Player Opener

Let’s play Hangman. You will be the host.

Rules for you (the host):

- Privately choose one secret existing English word (lowercase letters only). Do not reveal it unless I ask to reveal it.
- I have 6 lives.
- After each of my guesses, reply including both:
 - 1) The current pattern using underscores and revealed letters only (e.g., "_ a _ e _").
 - 2) The number of remaining lives.
 - 3) The guessed letters.
- Be consistent with the rules and do not restate or hint the secret.

I will begin guessing letters now.

Player Guess

My next guess is the letter "{letter}". Is it in the secret word?

Candidate Verification

Is the secret word "{word}"? Answer only with a single word: yes or no.

Reveal Request

Please reveal the secret word now. Respond with exactly one lowercase English word, with no punctuation or extra text.

Candidate Secret Generation (LLM Helper)

From the Hangman transcript below, infer the constraints and return exactly {max_n} plausible secret words (lowercase, no spaces or hyphens) that are all consistent with the dialogue so far.

Constraints to respect when proposing each word:

- Word length must match the most recent pattern (count underscores and revealed letters).
- Letters confirmed PRESENT must appear in the word in the exact positions indicated by the pattern.
- Letters confirmed ABSENT must not appear anywhere in the word.

- Avoid duplicates; return distinct words only.
Return ONLY a JSON array of strings, e.g., ["apple", "angle"]. No extra text.
<transcript> {transcript} </transcript>

Diagnosis Simulator.

Player Opener

You are the attending physician hosting a diagnostic session.

Rules for you (the host/patient):

- Privately choose one random, realistic, but non-obvious medical condition (the hidden condition). Do not reveal it unless I ask to reveal it.
- I (the resident) will ask short yes/no questions about the presence of specific symptoms, antecedents, signs, or findings.
- After each of my questions, reply including:
 - 1) Answer: yes or no.
 - 2) A concise running summary (optional, 1-2 lines) that remains consistent across turns.
- Be consistent and do not leak the medical condition unless I ask to reveal it.

I will begin now with initial questions.

Player Question

Do you have {symptom}?

Candidate Verification

Is the hidden condition {diagnosis}? Answer only with a single word: yes or no.

Reveal Request

Please reveal the medical condition now. Respond with the medical condition name only. Do not include any extra text.

Candidate Secret Generation (LLM Helper)

From the diagnostic transcript below, infer constraints and return exactly {max_n} plausible medical conditions that are all consistent with the dialogue so far.

Constraints to respect when proposing each condition:

- The condition must be compatible with the signs, symptoms, and labs discussed.
- Avoid duplicates; return distinct conditions only.

Return ONLY a JSON array of strings (conditions may be multi-word), e.g., ["influenza", "pulmonary embolism"]. No extra text.

<transcript> {transcript} </transcript>